

MACE: A Multi-core Agentic Co-design Engine for Autonomous OpenPiton Bring-Up

Rana Umar Nadeem, Samrah Mumtaz, and Muhammad Imran
CHIA A3 Hackathon Submission, September 2026

Abstract—MACE is an agentic AI system that builds and verifies multicore hardware on its own, running on the CHIA framework. Phase 1 adds `chia_openpiton`, a CHIA adapter exposing OpenPiton’s `configure`, `build`, `simulate`, `run`, and `collect` operations, validated with 208 tests. Phase 2 adds the MACE loop: a planner breaks a hardware objective into tasks, agents run them in parallel, and a deterministic integrator gates every change against Verilator simulation before accepting it. A failure-analysis agent diagnoses failures and the planner replans around them.

We tested MACE against two baselines, manual mesh scaling and a one-shot LLM prompt, on 2x2 and 4x4 meshes, with every result gated by Verilator simulation of the RTL. We added PicoRV32 as a third core, resolving several RTL bugs that kept it from passing under Verilator, and traced what kept multi-tile meshes from passing, including a Verilator 5 scheduling bug that makes CVA6’s cache adapter drop invalidations. With those issues addressed, MACE’s loop passes 2x2 and 4x4 meshes of both Ariane and PicoRV32, while the one-shot baseline fails every configuration it attempts. The artifact pins its toolchain in a Nix flake, entered with a single `nix develop`.

1. Introduction

Current agentic coding systems are typically benchmarked on isolated tasks or toy problems [1], [2], which fails to capture the complexity of actual hardware construction. Bringing up a multicore system means navigating a large RTL codebase (OpenPiton [3] spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a fragile toolchain (version skew, cross-compilation bugs, checkout-specific issues), and verification that costs minutes to hours per attempt rather than milliseconds.

MACE is designed for this high-friction environment. Given a hardware objective, such as “bring up a 2x2 mesh” or “verify workload W passes,” MACE plans, implements, builds, simulates, diagnoses failures against Verilator [4] output, and iterates. All validation in this paper is gated by Verilator simulation, not by an agent’s self-report.

1.1. Features

MACE contributes:

- `chia_openpiton` (Sec. 3): a CHIA adapter for OpenPiton with a full `configure/build/run/collect` interface, built the same way as CHIA’s `gem5` and `ESP` adapters, so it can be upstreamed directly.
- The MACE loop (Sec. 4): planning, parallel task execution, deterministic integration, and LLM-driven failure triage, tested end to end against full Verilator runs, with cost, time, and task metrics.
- A third core, PicoRV32 (Sec. 5), brought up by fixing three bugs in its upstream OpenPiton integration.
- An interactive CLI and two further capabilities (Sec. 6): Verilator code coverage as a second verification axis, and a second working LLM backend.

2. Background

CHIA [5] provides the agent, tool, and dispatch layer this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and Ray-based [6] remote dispatch and caching. OpenPiton [3] is Princeton’s open-source, silicon-proven manycore research framework. Tiles connect through a 2D-mesh network-on-chip and speak a custom MESI-coherence protocol (L15, descended from OpenSPARC T1’s PCX/CPX bus) to reach the NoC. OpenPiton supports three tile cores, the latter two added through its BYOC framework [7]: OpenSPARC T1 (`sparc`), Ariane/CVA6 [8] (`ariane`, RV64GC, coherent), and PicoRV32 [9] (`pico`, RV32I, non-coherent). Before this project, CHIA had no OpenPiton adapter.

Related work. LLM-based hardware design has mostly been evaluated at module scale. VerilogEval [2] scores single-module Verilog generation, ChipNeMo [10] adapts LLMs to chip-design tasks, and AutoChip [11], RTL-Fixer [12], VerilogCoder [13], and MAGE [14] feed compiler or simulation results back to one or more agents to repair generated RTL. In software, SWE-bench [15] moved evaluation from isolated functions [1] to repository-level issues. MACE applies the same agent-with-feedback idea at system scale: rather than writing one module against a small testbench, it plans and verifies bring-up of an existing manycore, gated by whole-system Verilator simulation.

3. Phase 1: The OpenPiton Adapter

Design. The adapter, `OpenPitonWorkspaceNode`, is a CHIA `CollocatedNode` whose resource-pinned remote

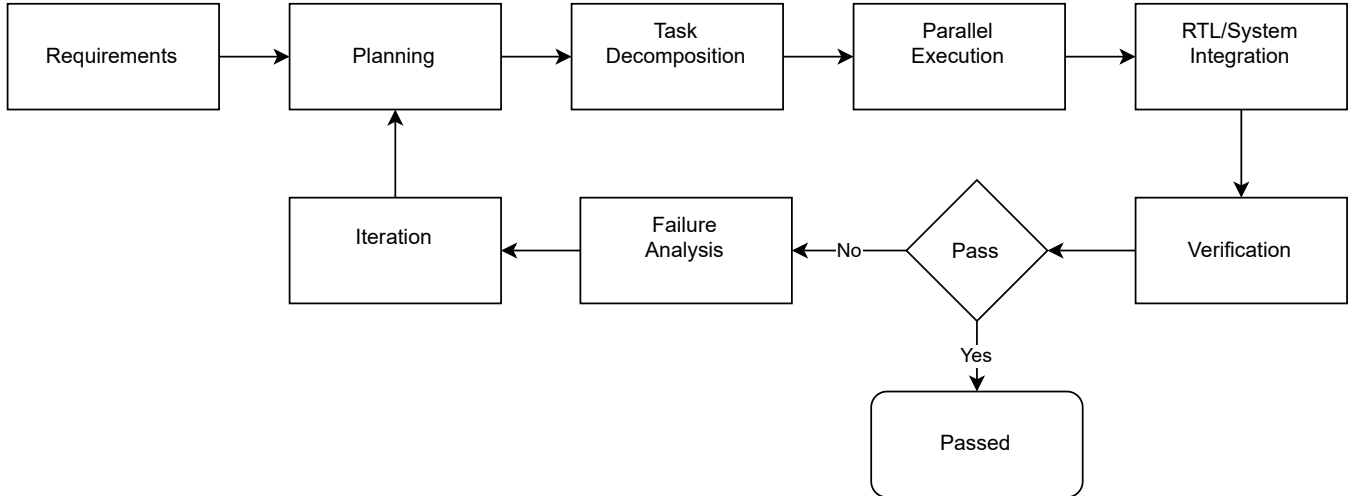


Figure 1. The MACE pipeline. Verification gates every integrated change against Verilator simulation. The run ends when every gate workload passes; any failure triggers failure analysis and a replan.

functions configure, build, run, regress, collect results, and clean. OpenPiton’s template preprocessor writes generated `.tmp.v` files into the source tree on every build, so the adapter gives each worker its own checkout, reserved through one `openpiton` resource unit, instead of locking a shared one. Pure parsers read build and run verdicts from OpenPiton’s transcripts (`sim.log`, `status.log`), using the strings its RTL monitors emit.

Environment hardening. The build surfaced four infrastructure bugs, fixed in one idempotent patch script: `binutils 2.38+` splitting `zicsr/zifencei` out of base RV64I, which breaks the boot ROM’s assembly; GCC 15’s C23 default breaking a legacy function call; broken git symlinks on a Windows-mounted checkout; and CRLF shebangs in 66 files. A Nix flake [16] pins the toolchain, including a Verilator release (5.052) that avoids two Verilator bugs this RTL triggers.

Build-result caching. `build()` skips the simulator when a matching successful-build marker exists: rebuilding an identical configuration dropped from 104s to 0.00s, so an iteration that only changes the test never rebuilds.

Results. 208 tests that need no OpenPiton checkout (pure-unit, and live-Ray with a stub simulator) all pass, and acceptance tests with full Verilator builds pass, including two independent checkouts building in parallel: 176s versus 324s serial. One acceptance test, 2x2 Ariane on a GCP worker, remains blocked by upstream infrastructure dependencies (Sec. 9).

4. Phase 2: The MACE Loop

Pipeline. Figure 1 shows the loop. A planning agent turns a `MaceSpec` (core, target mesh, gate workloads, objective, budget) into a task DAG, and workers dispatch ready tasks in parallel through Ray, each pinned to its own checkout. A deterministic integrator, with no LLM, applies accepted diffs in dependency order, then builds and runs

every gate workload; each workload’s *verdict*, parsed from simulator output, must read `pass`. On any failure, a failure-analysis agent with read-only tools (log grep, file collection, a fixture diff, and an `objdump` symbol check) diagnoses the transcript, and the planner replans around its proposed fix.

Gate workloads surface a memory-visibility bug. Writing the gate workloads (barrier-and-atomic-counter, producer/consumer, hart-id scatter/gather) showed that a plain `volatile` read of a shared value was not reliably visible from the same hart shortly after the write, while an atomic fetch-and-add-zero read always was. Every gate workload now reads shared values through this `atomic_read()` helper.

Resource enforcement. MACE checks all three `Budget` dimensions (iterations, USD, wall-clock) before each iteration starts, and verifies gate-workload checksums before any LLM call or checkout access, so a task cannot tamper with its own grading.

Deterministic replay. MACE cache-tags LLM, build, and run decisions and re-serves them at no token or compute cost. Through CHIA’s cache, it replays *decisions* (response text, pass/fail artifacts), while git tracks the source edits agents make.

5. A Third Core: PicoRV32

Integrating arbitrary core RTL into OpenPiton means navigating fundamental architectural differences rather than wiring up a standard bus. OpenPiton has no reusable core-to-NoC bridge: under BYOC, each core reaches the L15 through its own transducer, and Ariane’s AXI4 output goes unused because a hand-written encoder taps CVA6’s internal pre-AXI cache-miss stream. The dividing line is whether the core has its own coherent cache: a new core with one, as Ariane has, needs new coherence-adaptor RTL. PicoRV32 does not, and a complete upstream transducer

(`pico_l15_transducer.v` and friends) already exists, although OpenPiton’s CI builds it under Verilator without running it.

Results. The adapter extension (a new `Literal` entry, two `sims` flags, mirrored tests) landed with full unit coverage and no regressions to the `ariane` and `sparc` paths, but the first `pico` runs ended at `maxcycles`. Comparing the binary’s symbols against `symbol.tbl` and its transcript against a passing `Ariane` run ruled out a build or toolchain problem and placed the stall at the core never reaching its trap address.

Waveform tracing found three RTL/testbench bugs: PicoRV32’s `resetn/booted` self-boot gate, the manycore monitor’s `active_thread` tracking for `pico`’s tile, and the SRAM model’s power-on BIST self-clear, which discarded `pico`’s first memory writes. With all three fixed, PicoRV32 **passes** under Verilator on 2x2 and 4x4 meshes, every tile reaching `Hit Good trap` on `addi.S`. MACE surfaced these bugs because its gate distinguishes a successful build from a correct run. `barrier_atomic.c` never runs on `pico`: its OpenPiton integration has only an assembler, with no C compiler or `crt.S/syscalls.c`, so the diag never compiles. This is a toolchain gap rather than an RTL bug, and the upstream `amoad_w.S` test confirms that `pico`’s atomic hardware works.

6. Additional Capabilities

MACE features a standard EDA-style interactive shell (modeled on Yosys/OpenROAD) that validates core compatibility up front to prevent invalid integration attempts. To provide a second verification axis beyond pass/fail, we patched the OpenPiton testbench to call Verilator’s coverage-write API, yielding metrics such as 35.00% line coverage for a 2x2 `Ariane` run. The agentic loop is also backend-agnostic, completing full passing runs unmodified against Google Vertex Gemini and OpenCode.

7. Evaluation

We compare three approaches on 2x2 and 4x4 meshes: (a) manual mesh scaling with a time log, (b) a single LLM prompt for a configuration applied once with no tools, verification, or retry, and (c) the full MACE loop, with every LLM call on Gemini 2.5 Flash (Table 1). `Ariane` runs `barrier_atomic.c`; `pico` runs `addi.S`, since its integration has no C toolchain; `SPARC` does not yet run under our Verilator (Section 9). All three approaches get `pico`’s `CONFIG_DISABLE_BIST_CLEAR` define: (a) sets it by hand, and the objectives for (b) and (c) name it; (b)’s config line accepts RTL defines, and (c)’s planner requests one through a new `CONFIG_RTL`: directive, which adds it on top of the mesh’s defaults.

(b) fails every configuration. In three of four, `PitonConfig`’s construction-time validation rejected the LLM’s config before any build: it set the L1.5 size to zero, which every OpenPiton tile needs, and (b) has no retry path.

TABLE 1. VERILATOR VERDICT AND END-TO-END WALL TIME PER APPROACH. ALL RUNS SHARE ONE BUILD CACHE, SO AN APPROACH PAYS FOR A BUILD ONLY WHEN IT PICKS A NEW CONFIGURATION; (A) COUNTS MACHINE TIME ONLY. [†]INVALID CONFIGURATION (L1.5 SIZE 0), REJECTED BEFORE ANY BUILD.

Core	Mesh	(a) Manual	(b) One-shot	(c) MACE
ariane	2x2	pass, 33.9s	fail, 10.4s [†]	pass, 750.1s
	4x4	pass, 271.9s	fail, 3820.9s	pass, 250.9s
pico	2x2	pass, 9.8s	fail, 12.0s [†]	pass, 70.7s
	4x4	pass, 53.9s	fail, 13.4s [†]	pass, 100.6s

On `pico` it included the named define both times. At `ariane` 4x4 it chose oversized caches, including a 4 MB L2 per tile: the build took 35 minutes and the simulation timed out with no tile finished.

Mesh scaling to 2x2 and 4x4. Manual scaling (a) first stalled at 2x2 and never reached 4x4, for two reasons. Under Verilator 5, a known scheduling bug [17] makes CVA6’s L1.5 adapter drop cache invalidations, so the plain-load exit barrier in `syscalls.c` never sees other tiles’ updates; OpenPiton pins a CVA6 that predates the upstream workaround [18]. Porting that workaround alone passes 2x2 with the unmodified barrier (4 of 4 tiles, 24s), while the original adapter times out with one tile done. Separately, the monitor’s 32-bit `finish_mask` capped verification at 8 tiles. Our runs poll the barrier with atomic reads rather than port the adapter fix, and widen the mask. With both in place, manual runs pass both sizes, and the loop planned and verified both meshes, every tile confirmed, with no changes to its planning or verification logic (Table 1): 2x2 in 750.1s across four tasks, including an L1D-associativity variant the planner chose to verify, and 4x4 in 250.9s on a cached build. On `pico` it passes in a single task at both sizes.

Recovery. The detect, diagnose, and replan cycle is exercised only at the mechanism level, with a stub that fails once and then passes; no passing run above needed it. Its one live failure, a `pico` 4x4 build cached before the `finish_mask` fix, went unrecovered: a build ID covers configuration but not source edits, so all three replans reused the stale build while triage blamed the RTL. A clean rebuild passed.

8. Cost

All experiments behind this paper cost \$37 of a \$300 GCP credit allocation, covering simulation compute and LLM inference; development used a \$20/month Claude subscription, outside that budget. A single loop run is cheap. At 2x2, an `ariane` run costs about \$0.12 (\$0.04 of LLM tokens and \$0.08 of compute) and a `pico` run about \$0.02, while a `sparc` run that exhausts its three-iteration budget would cost an estimated \$0.25, mostly for its one-time build. We price compute as `e2-highmem-8` time (\$0.36/hour) over each run’s wall time, and tokens at Gemini 2.5 Flash’s \$0.30 and \$2.50 per million input and output tokens. MACE’s own cost counter reads zero because CHIA’s Vertex backend reports no price and omits thinking tokens, more than half of the

billed output here, so we measured tokens by re-sending representative planner and task prompts.

9. Limitations and Open Issues

SPARC under Verilator 5. OpenSPARC T1 builds but does not run in our environment: the core never receives the I/O bridge’s power-on wake-up interrupt, so no thread starts and the run hits `max_cycle` with no verdict. OpenPiton’s exact CI recipe, with its CI-verified `princeton-test-test.s`, stalls the same way under our Verilator 5.020 (with `--no-timing`, which Verilator 5 requires here), while CI uses Verilator 4, so the stall is environmental; its root cause remains open. `barrier_atomic.c` would not run on T1 regardless: it uses RISC-V inline assembly, and T1 has no C diag environment.

GCP dispatch. A task needing a tailnet-relayed GCP worker first never ran because CHIA sets three proxy environment variables during `chia up` that `chia job submit` inherits but our manually launched drivers did not; exporting them took the task from a six-minute hang to a two-second run. A later attempt on the same topology hung again with that fix in place, and relay logs plus a raylet `strace` show no outbound activity, pointing at Ray’s scheduling under a proxied topology. That follow-up remains open.

Future work: generality over one-off adapters. All three cores here already had upstream OpenPiton/L15 adapters (SPARC’s CCX transducer, Ariane’s L15 adapter in CVA6, PicoRV32’s transducer); a core without one, such as Ibex or Rocket Chip, needs it written first. The existing three share a shape: a per-core RTL directory, a transducer between the core’s memory interface and L15, and a core-select arm in the tile template. The next step is turning that shape into a template and adding 2-4 more cores, with MACE’s loop verifying each iteration of the adapter RTL in Verilator while a hardware engineer writes it. A second direction is a dashboard over the run database, showing dispatch, build, and verification live instead of through raw trace files.

10. Conclusion

MACE shows that an agentic loop can plan, build, and verify multicore hardware, with every result gated by RTL

simulation. Along the way we fixed environment and RTL bugs and added a third core, and the loop then passed 2x2 and 4x4 meshes of both Ariane and PicoRV32. The next steps are a reusable adapter template for new cores and a recovery cycle proven on live failures.

References

- [1] M. Chen *et al.*, “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [2] M. Liu *et al.*, “VerilogEval: Evaluating large language models for Verilog code generation,” in *Proc. ICCAD*, 2023.
- [3] J. Balkind *et al.*, “OpenPiton: An open source manycore research framework,” in *Proc. ASPLOS*, 2016, pp. 217–232.
- [4] W. Snyder *et al.*, “Verilator,” <https://verilator.org>.
- [5] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” arXiv:2606.27350, 2026.
- [6] P. Moritz *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proc. OSDI*, 2018, pp. 561–577.
- [7] J. Balkind *et al.*, “BYOC: A ‘Bring Your Own Core’ framework for heterogeneous-ISA research,” in *Proc. ASPLOS*, 2020, pp. 699–714.
- [8] F. Zaruba and L. Benini, “The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology,” *IEEE Trans. VLSI Syst.*, vol. 27, no. 11, pp. 2629–2640, 2019.
- [9] C. X. Wolf, “PicoRV32 – a size-optimized RISC-V CPU,” <https://github.com/YosysHQ/picorv32>.
- [10] M. Liu *et al.*, “ChipNeMo: Domain-adapted LLMs for chip design,” arXiv:2311.00176, 2023.
- [11] J. Blocklove *et al.*, “Automatically improving LLM-based Verilog generation using EDA tool feedback,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 6, 2025.
- [12] Y. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically fixing RTL syntax errors with large language model,” in *Proc. DAC*, 2024.
- [13] C.-T. Ho, H. Ren, and B. Khailany, “VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool,” in *Proc. AAAI*, 2025.
- [14] Y. Zhao *et al.*, “MAGE: A multi-agent engine for automated RTL code generation,” in *Proc. DAC*, 2025.
- [15] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [16] E. Dolstra, M. de Jonge, and E. Visser, “Nix: A safe and policy-free system for software deployment,” in *Proc. LISA*, 2004, pp. 79–92.
- [17] G. Lore, “Partial variable assignment leads to incorrect ordering/lack of circular logic detection,” Verilator issue #5829, <https://github.com/verilator/verilator/issues/5829>, 2025.
- [18] —, “Workaround for Verilator ordering issue in OpenPiton cache adapter,” CVA6 pull request #2809, <https://github.com/openhwgroup/cva6/pull/2809>, 2025.